CSE 1242/1142 - COMPUTER PROGRAMMING II

TERM PROJECT

DUE DATE: 12/05/2025 (No extension)

In this project, you will implement a tower defense game using JavaFX where players:

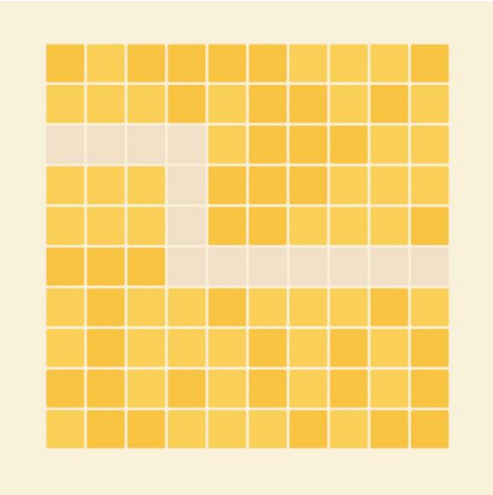
- Place **towers** on a grid to defend against waves of enemies.
- Manage resources to buy towers and stop enemies from reaching the end of a path.
- Destroy enemies using bullets, lasers and missiles, triggering **explosion effects** on elimination.
- Survive increasingly difficult waves of enemies.
- **Win** by defeating all waves; **lose** if 5 enemies reach the endpoint.

Project Details

Game Maps

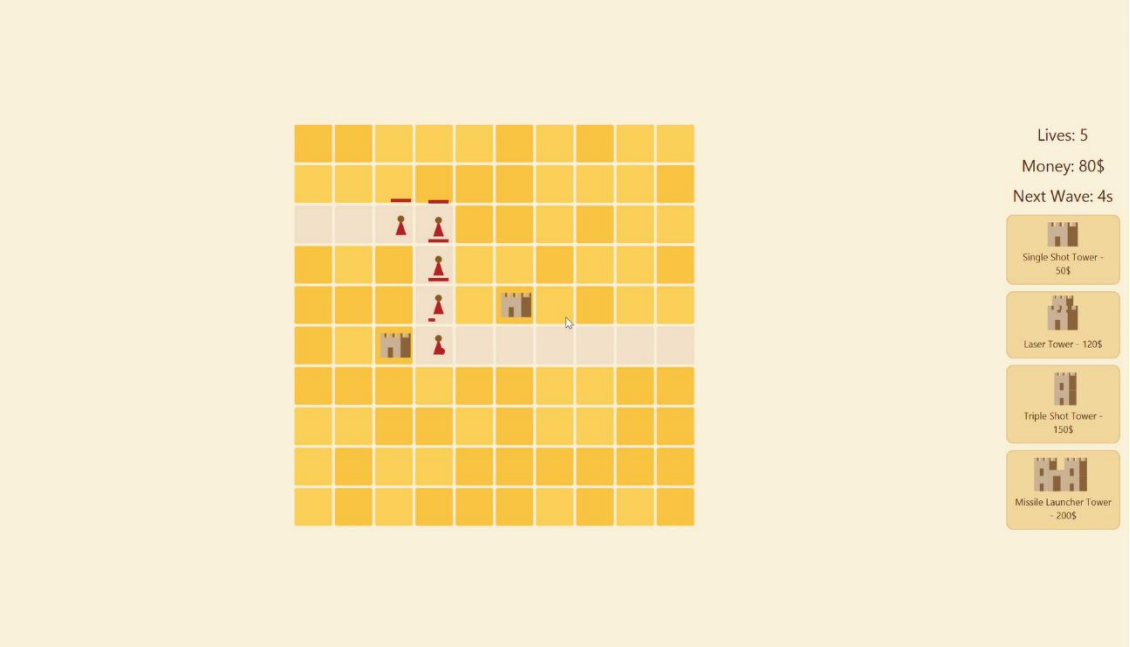
Grid System: The game map consists of a grid of cells where players place towers. Path cells are predefined for enemy movement. In here, the grey cells are the path where enemy moves through. Other yellow cells are the parts where the towers can be placed. There are two different yellow tones that are assigned randomly to the cells. When a level starts, an animation plays to load the game map grid. This effect is obtained by animation every cell's scale x and scale y. There should be delays between each cell animation. Complete animation can be seen in the gameplay video.

An example grid:

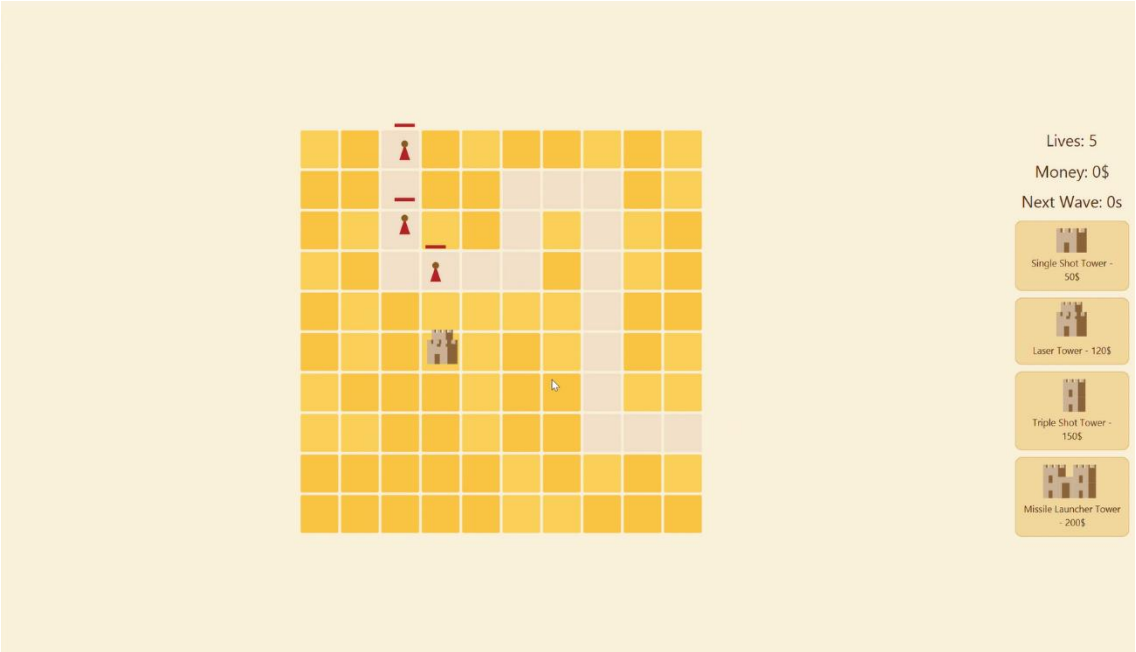


Level Maps:
5 level maps will be given. Those 5 maps are shown below.

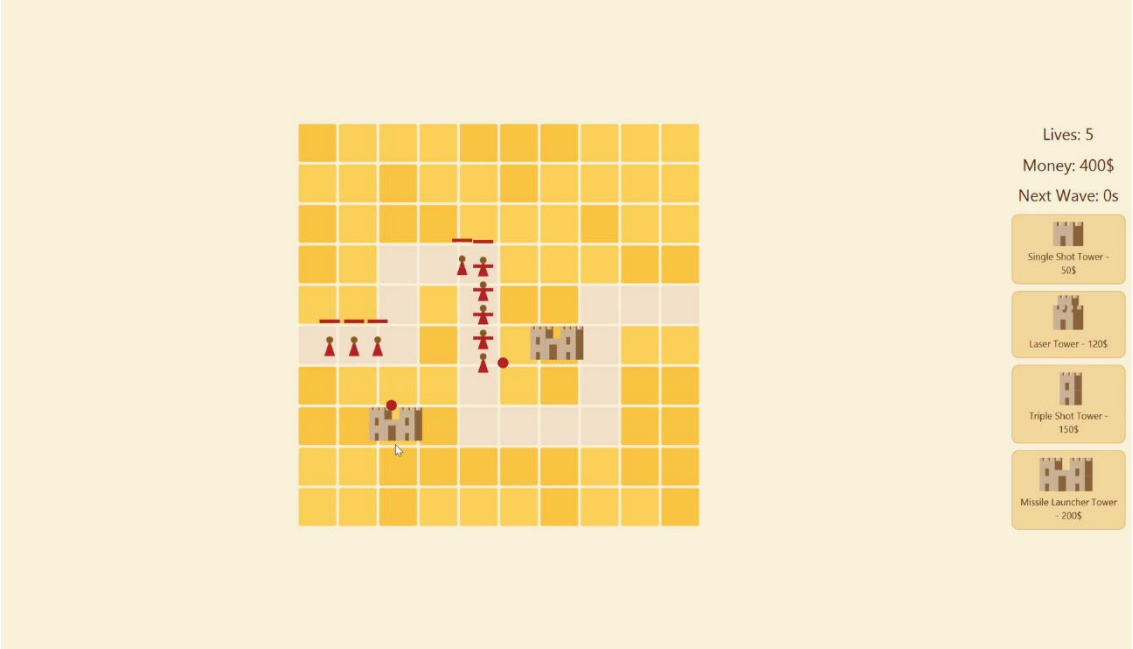
- **Level 1:** 10 x 10 Grid



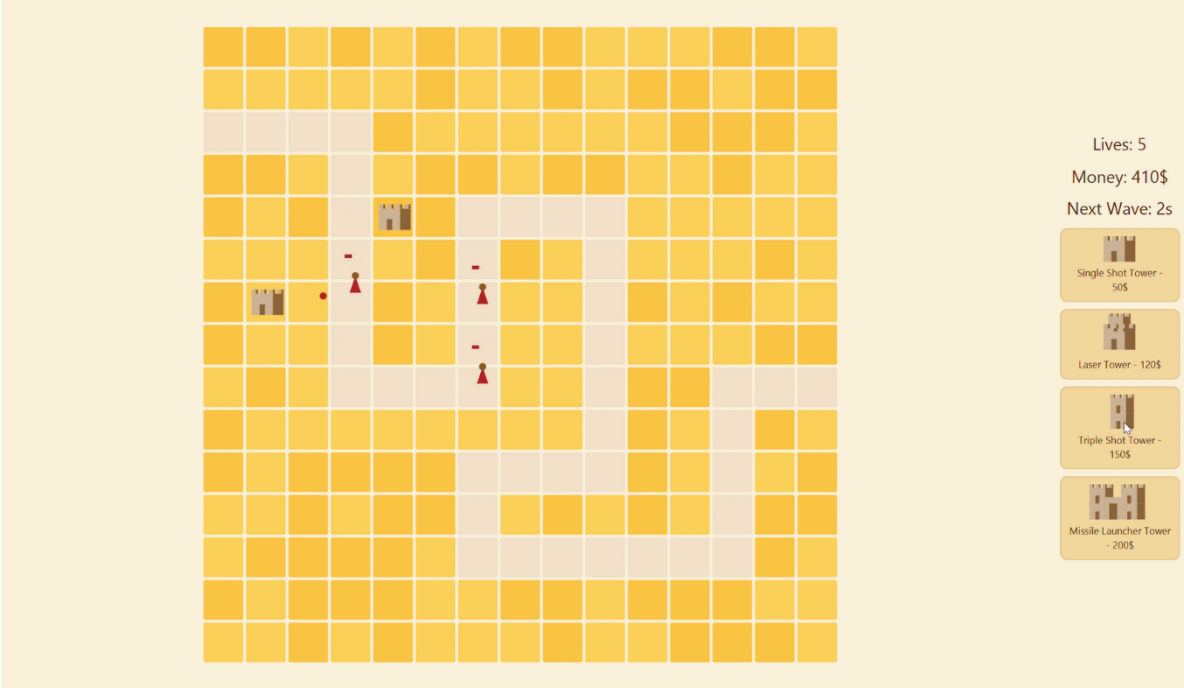
- **Level 2:** 10 x 10 Grid:



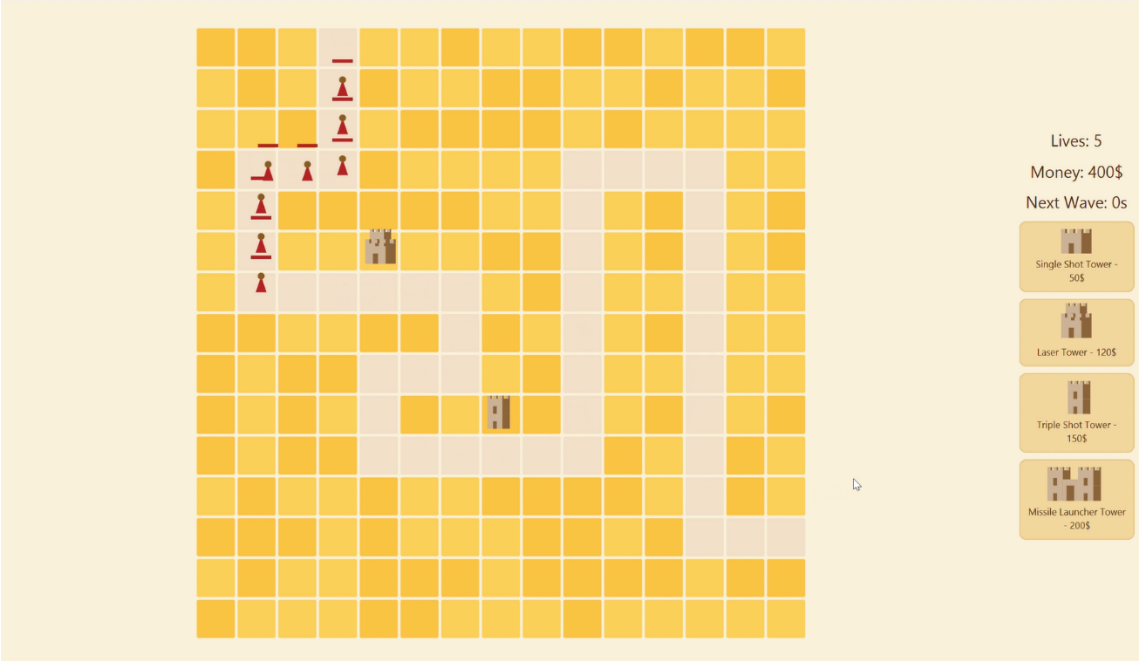
- **Level 3:** 10 x 10 Grid



- **Level 4:** 15 x 15 Grid



- **Level 5:** 15 x 15 Grid



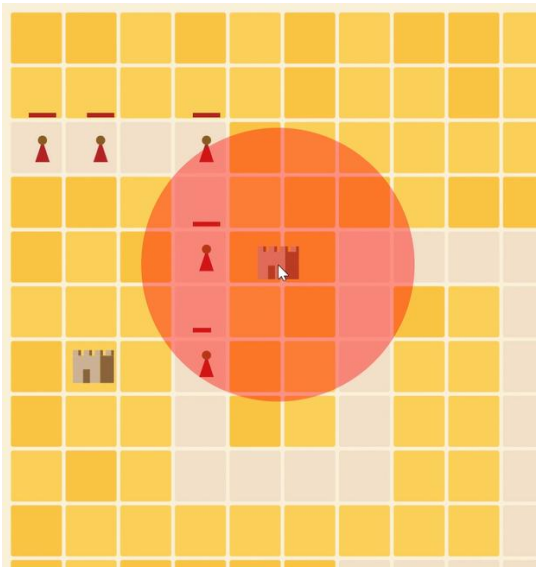
Enemies

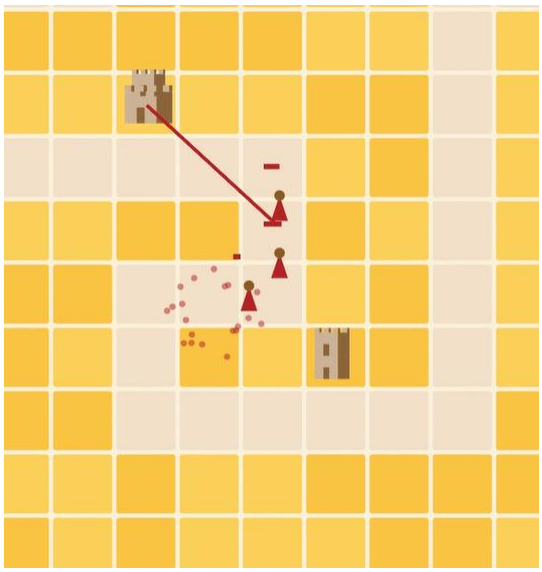
Enemies will follow the gray path given for the level. The goal of the enemies is to reach the end of the path while the towers try to shoot them. Each enemy has a health bar showing how much damage it can take before it explodes. Health bars should be updated when the enemy gets damaged. When there is no health remaining, the enemy dies and explodes. If an enemy reaches the end of the path, remaining lives of the player gets decreased by one. If an enemy dies, the player gains 10$ and the money label gets updated.

Enemies can be drawn with Circle and Polygon shapes. You can also use an image asset to draw enemy shape.

The explosion effect when an enemy die should be implemented by animating 20 particles drawn as small circles. A random angle and distance should be determined for each particle and they should be animated going in that direction for 500 milliseconds. Gameplay video shows more details about enemy movement and explosion.

In the figure on the left, enemies are shown as moving red objects and the area of effect of the tower is shown as a red ring. In the figure on the right, the explosion effect when a tower shoots an enemy is shown.





Towers

Towers automatically target and shoot enemies within range. Player places towers on the game map by dragging them from the side menu. Each tower has their own price and the player can only buy them if s/he has enough money. Towers can be sold by dragging them out of the map. Also, the player can change the position of the towers to create better strategies during the gameplay. Towers has a range radius and can only shoot enemies within range. A red circle gets drawn when the towers get dragged to show the range. Only one tower can be placed in a cell. The player should not be able to place towers on top of each other. Also, the player should not be able to place the tower if s/he does not have enough money to buy it.

There are 4 different tower types:

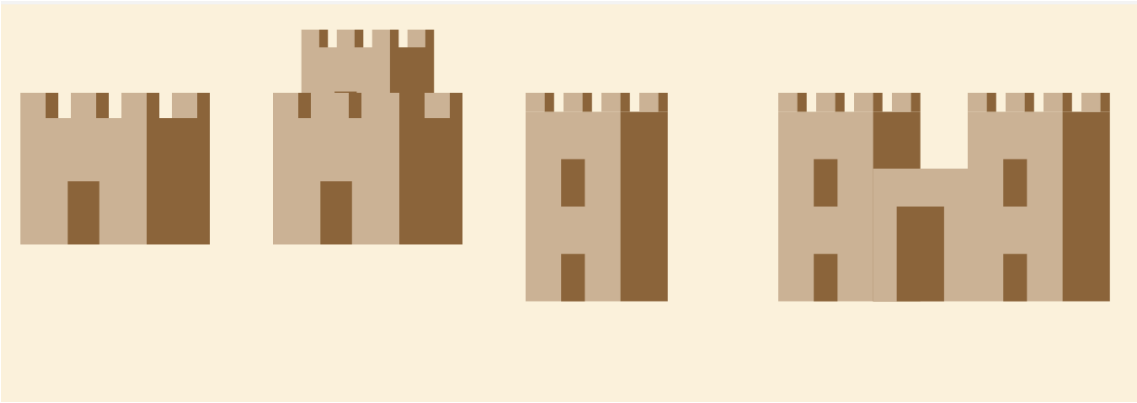
- **Single shot tower:** It is the tower that shoots a single bullet to enemies in range. A bullet gives 10 damage to the enemy and an enemy has 30 health. However, you can change these parameters for optimal gameplay. The bullet should be animated from the tower position to the enemy position. The tower shoots the nearest enemy within the range first and continues to shoot it until it leaves the range or dies. This type of tower can only shoot one enemy at a time.

- **Laser tower:** It locks a target in its range with a red laser and reduces its health continuously by giving smaller damage at each time frame. You can implement the laser shooting feature with an Animation timer and you can use your own time interval and damage amount. The laser tower targets all the enemies within range unlike the single shot tower. It continues to track the enemies with laser until they leave the range or die. The laser can be drawn as a red line from the tower to the enemy.

- **Triple shot tower:** This tower type is similar to the single shot tower. However, instead of targeting one enemy at a time, it shoots three nearest enemies at the same time. The damage it gives to the enemies is also more than the single shot tower.

- **Missile launcher tower:** This tower launches a missile. The missile targets the nearest enemy like the single shot bullet. However, when the missile lands, it kills all enemies that are within the area of effect. The missile has its own effect radius. You can define your optimal missile range. Missile moves slower than a bullet and drawn as a bigger circle.

Each tower has different images. You need to draw the tower images using Rectangle shapes. Following image shows all towers as a reference. It shows towers in the order given above.



UI Elements



- Red bars on top of each enemy shows its heath status.
- Visual range indicators for towers.

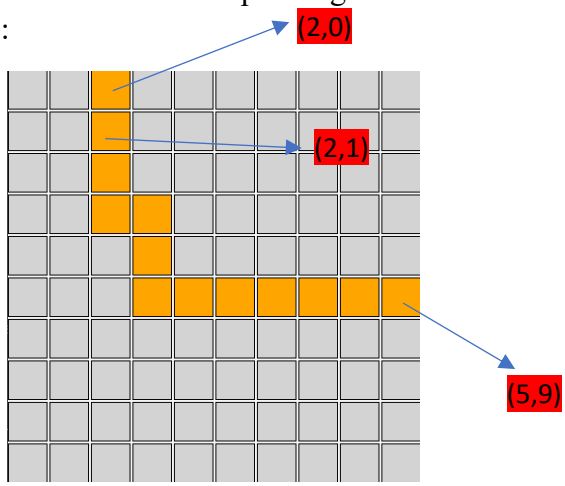
  A Panel that **contains current lives left**, **money** and a **timer** to next wave. Inside the panel, there are tower symbols where you can drag and drop each tower to intended position in the grid.

  - Lives: Initially, the player has 5 lives. For each enemy that reaches the end point, the lives decrease by one. If the number of enemies that reach to the end is 5, the game is over.
  - Money: Initially, the player has $100. For each enemy destroyed, the player gets $10. Each tower has a different price and can be bought via accumulating money.
    -Single Shot turret: $50
    -Laser Turret: $120
    -Triple Shot Turret: $150
    -Missile Launcher Turret: $200

Wave/Next Wave: the term “Wave” stands for an enemy group. There are different number of waves in each level. Each wave includes different number of enemies. There are time delays between each wave so that player can have enough time to create a strategy and buy towers until the next wave. Next Wave timer shows how many seconds left until the next wave. Details of the wave mechanic is specified in the “Wave Mechanic” section.

Level Parsing

Different levels of the game should be created by reading the data from corresponding level file. The following information can be read from the level file:



- **Level Map**: The level file starts with lines defining grid size and path cells. It always starts with the “WIDTH” and “HEIGHT” properties of the grid. Subsequently, path of the enemies is referred as their row and column indexes. For instance, the grid cell at 2nd line and 0th column will be the start of the path and it’s directly connected to the cell at 2nd line and 1st column. By reading all the lines between HEIGHT line and the WAVE_DATA line in the file, all the indices of the path can be gathered in order.
- **Wave properties:** Remaining lines after the WAVE_DATA: line corresponds to the wave properties of the given level. Detailed explanation is given in the Wave Mechanic section.

Example format:

Main.java | map1.txt | map3.txt | map4.txt | map5.txt | map2.txt

```
1 WIDTH:10
2 HEIGHT:10
3 2,0
4 2,1
5 2,2
6 2,3
7 3,3
8 4,3
9 5,3
10 5,4
11 5,5
12 5,6
13 5,7
14 5,8
15 5,9
16 WAVE_DATA:
17 5, 1, 2
18 8, 0.5, 5
19 12, 0.3, 5
```

Wave Mechanic

In the game, waves have three main parameters:

Enemy Count: Determines how many enemies will spawn in the wave.

Time Delay Between Enemies: Specifies the time interval (in seconds) between the spawning of consecutive enemies within the same wave.

Time Delay Before the Wave Starts: Defines the waiting time (in seconds) before the wave begins.

These three parameters will be read from a file. The "WAVE_DATA:" header marks the beginning of the wave definitions. Each line following this header represents a single wave and contains the three parameters in order. The game will use this data to generate the waves accordingly. **Details of the WAVE_DATA spec:**

{5, 1, 2}: Wave 1: 5 enemies, 1 second between enemies, 2 second delay before start

{8, 0.5, 5}: Wave 2: 8 enemies, 0.5 second between enemies, 5 second delay

{12, 0.3, 5}: Wave 3: 12 enemies, 0.3 second between enemies, 5 second delay

{20, 0.3, 5}: Wave 4: 20 enemies, 0.3 second between enemies, 5 second delay

Game Mechanics

Enemy Movement

Enemies follow predefined path. They should be spawned from the first cell in the path. They should move with constant speed. You may adjust the enemy speed as you like.

Tower Placement and Shooting

- Players drag towers from the right panel and drop non-path grid cells to place towers.
- Towers automatically target the nearest enemy using Euclidean distance calculations.

Wave Spawning

- Waves increase in difficulty (e.g., more enemies, faster spawns).
- Note that enemy health levels are the same for each wave and is determined 30. However, you may adjust this level as you like.
- **Timer to Next Wave**: There is a short waiting period between waves. There is a timer to show this period.

Currency

- Each tower has a different price. The player increases the amount of money as the towers eliminate enemies, and after collecting enough money for other towers, the player can spend that money to get them as well. For each level change, player can keep the money from the previous level.

Lives

- For each enemy that reaches to the end consumes a live from the player. Initially, the player has 5 lives.

Win and Lose Conditions

- **Win**: Survive all waves.
- **Lose**: 5 enemies to reach the endpoint without being eliminated.
- **Level Change:** After surviving all the waves on the map, the player can move on to the next level.

Bonus Features

- **Tower Upgrades**: Add damage, range, or fire rate upgrades. Also, physically reflect the change on the tower either (Change the color, size etc.).
- **Multiple Enemy Types**: Fast, armored, or flying enemies.
- **Assets**: You can change the appearance of cells, towers and enemies by using external pictures.

  - Example Asset #1: https://free-game-assets.itch.io/free-fields-tileset-pixel-art-for-tower-defense
  - Example Asset #2: https://kenney.nl/assets/tower-defense-top-down

Submission Instructions

1. Submit a ZIP file named Student1ID_Student2ID_Student3ID_Project.zip containing:
    - Source code with detailed comments.
    - A 5-10 page project report.
2. Deadline: **12.05.2025 23:59**.

Demo Sessions

- Demonstrate gameplay, tower placement, and wave mechanics in a 15-minute session.

Project Report Requirements

1. **Cover Page**: Title, group member details, course information.
2. **Problem Definition**: Describe the tower defense game in 1-2 paragraphs.
3. **Implementation Details**:
    - UML diagrams for core classes.
    - Explanation of enemy movement, turret targeting, and wave spawning.
4. **Test Cases**: Screenshots of gameplay with explanations.

Notes

- You cannot use Scene Builder for the project.
- **Code Comments**: Include your name, ID, and purpose of each class/method.
- **Plagiarism**: FF grade for copied work.
- **Late Submissions**: Not accepted.
- **You will be given a video on how to play the game.**